CAMPUS NAVIGATOR

PROJECT REPORT : DSA Lab

DEPARTMENT OF COMPUTER SCIENCES
BAHRIA UNIVERSITY - ISLAMABAD CAMPUS

SESSION: SUMMER – 2026

SUBMITTED TO: MA'AM LAIBA GUL
SUBMITTED BY: FARAH ASAAD KAYANI (01-134222-046)

Table of Contents

1. Background

1.1. Scenario

University campuses are made up of multiple buildings, labs, and shared resources — library counters, cafeterias, admin offices — spread across a wide area. Students often don't know the fastest way to reach a place, how busy a resource currently is, or how to retrace their last few steps. Campus Navigator is a console-based tool that stores campus locations, lets a student search and browse them, simulates a waiting queue at a busy resource, and finds a path between two buildings.

1.2. Links

- GitHub Link:
  https://github.com/farahkayani/campus-navigator

- Deployed Project Link:
  https://campus-navigator-iota.vercel.app/

1.3. Problem Statement

Without a simple system, students rely on guesswork to find locations, check availability, or reach shared resources efficiently. This project addresses that by building a lightweight, data-structure-driven console tool that organizes locations, supports searching and sorting, simulates real queues, and maps connectivity between buildings.

1.4. Objectives

1. Store and manage a fixed set of campus locations using arrays.

2. Track recently visited locations and allow undo of the last navigation step.

3. Let the user browse the full location catalog forward and backward.

4. Simulate a first-come-first-served queue at a busy resource.

5. Search locations quickly using a Binary Search Tree.

6. Model campus buildings as a graph and check if one location is reachable from another (BFS).

7. Sort locations alphabetically and by a numeric field using four different sorting algorithms.

2. System Design Overview

Each requirement from the brief is implemented using exactly one core data structure:

Requirement (Task)	Data Structure Used	Where in Code
Task 1 — Store campus locations	Array of struct Location	locations[MAX_LOC]
Task 2/3 — Visit log + Undo	Array-based Stack	visitStack[ ], pushVisit( ), undoLastVisit( )
Task 4 — Browse forward/backward	Array + index pointer	browseIndex, browseCatalog( )
Task 5 — Busy resource queue	Array-based circular Queue (FCFS)	resQueue[ ], enqueuePerson( ), servePerson( )
Task 6 — Search by ID / Name	Two Binary Search Trees	idRoot, nameRoot, searchById( ), searchByName( )
Task 7 — Reachability between buildings	Graph (Adjacency Matrix) + BFS	adjMatrix[ ][ ], checkReachability( )
Task 8/9 — Sort alphabetically / numerically	Bubble, Selection, Insertion, Merge Sort	bubbleSortByName( ), selectionSortByNameDesc( ), insertionSortByDistance( ), mergeSortByDistanceDesc( )
Task 10 — Menu-driven interface	Switch-based menu loop	runMainMenu( ) in Menu.cpp

The menu flow below shows how a student moves between the 10 features from a single central menu.

2.1. Structure-by-Structure Explanation

- **Array — The Master Catalog**

All 19 campus locations are stored in a fixed-size array of a Location struct (id, name, distance). Arrays give O(1) random access by position and form the backbone every other structure refers to by index, avoiding data duplication. loadSampleData() populates this array at program start (Task 1).

- **Stack — Recent Visits Log and Undo**

Every time a student visits a location, its index is pushed onto a fixed-size array-based stack (visitStack[], pointer visitTop). Because a stack is Last-In-First-Out, showRecentVisits()

naturally prints the most recently visited location first (Task 2). undoLastVisit() pops the top of the stack in O(1) time, satisfying the undo requirement (Task 3).

- **Index Pointer — Catalog Browser**

browseCatalog() treats the location array like a menu deck. A single integer, browseIndex, tracks the current position; Next and Previous commands increment or decrement it with boundary checks, letting the student page through the entire catalog (Task 4).

- **Queue — Busy Resource Simulation**

A fixed-capacity circular array (resQueue[], with qFront/qRear/qCount) models people waiting at a busy shared resource — the Library Front Desk. enqueuePerson() adds an arrival to the rear; servePerson() removes and serves the person at the front — a First-Come-First-Served simulation (Task 5). Circular indexing via modulo arithmetic lets the fixed array be reused indefinitely without shifting elements.

- **Binary Search Tree — Fast Search**

Two BSTs are built once at start-up: idRoot, keyed by numeric ID, and nameRoot, keyed by location name. searchById() and searchByName() walk down the appropriate tree, and the program reports the number of comparisons made, directly demonstrating the O(log n) average-case advantage of a BST over a linear scan (Task 6).

- **Graph (Adjacency Matrix) + BFS — Reachability**

Campus buildings are nodes in an undirected graph; walkway connections are edges, stored as a oolean adjacency matrix built by buildCampusGraph(). checkReachability() runs a Breadth-First Search from the source, recording each node's parent as it's discovered. If the destination is reached, the parent chain is walked backwards to reconstruct and print the actual path; if BFS exhausts all reachable nodes without finding the destination, the tool reports that no path exists (Task 7).

- **Sorting Algorithms**

Four classic sorting algorithms are implemented from scratch, each operating on a copy of location indices so the underlying array is never disturbed (Tasks 8 and 9):

  - Bubble Sort → alphabetical order (A–Z) by name.
  - Selection Sort → alphabetical order (Z–A) by name.
  - Insertion Sort → numeric order by distance from the entrance (nearest first).
  - Merge Sort → numeric order by distance from the entrance (farthest first).

Both numeric sorts use distance rather than a fabricated rating field — mirroring exactly how the two alphabetical sorts both use name, just in opposite directions.

- **Menu-Driven Interface**

runMainMenu() in Menu.cpp ties every structure together behind a single numbered menu (Task 10). Each option calls into the relevant module and always returns control to the main loop, so a student can move freely between browsing, searching, queueing, and sorting within one session.

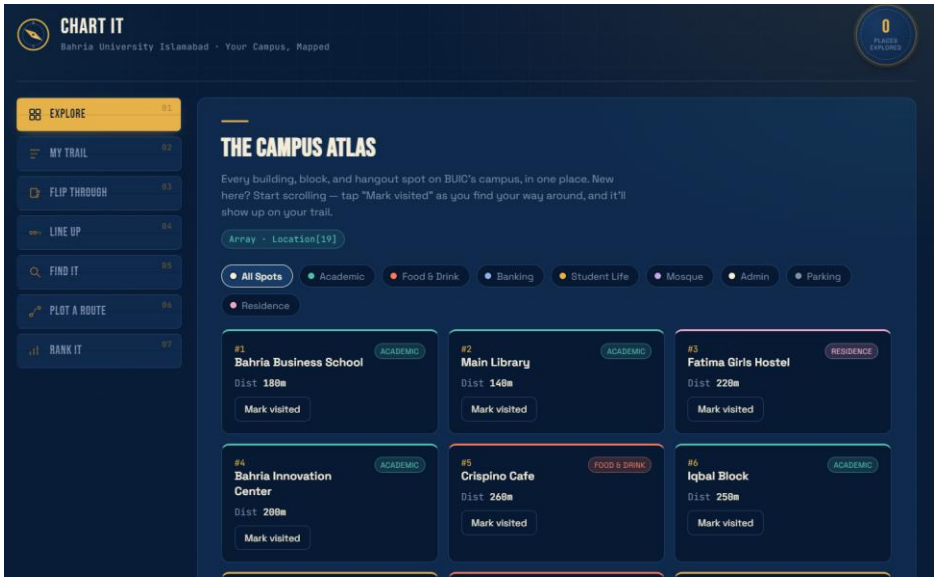
3. Complexity Summary

Operation	Data Structure	Time Complexity
Display all locations	Array	O(n)
Push / Pop a visit	Stack	O(1)
Enqueue / Serve	Queue	O(1)
Search by ID or Name	BST (balanced case)	O(log n) avg, O(n) worst
Reachability check	Graph + BFS	O(V + E)
Bubble / Selection Sort	Array	$O(n^2)$
Insertion Sort	Array	$O(n^2)$ worst, O(n) best
Merge Sort	Array	O(n log n)

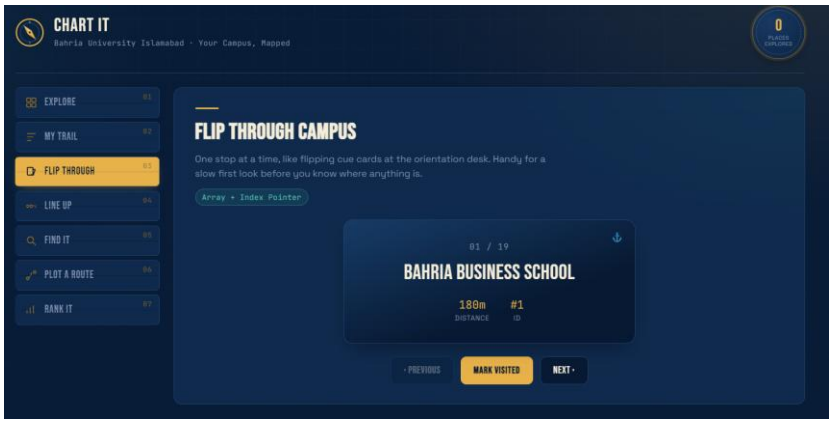
4. User Interface

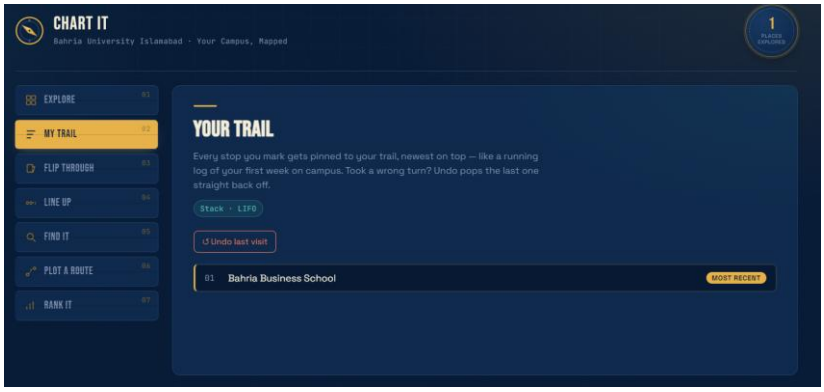
Below is a transcript of an actual run of the compiled program, demonstrating every feature end-to-end with the real BUIC dataset.

- **Displaying all locations (Task 1)**

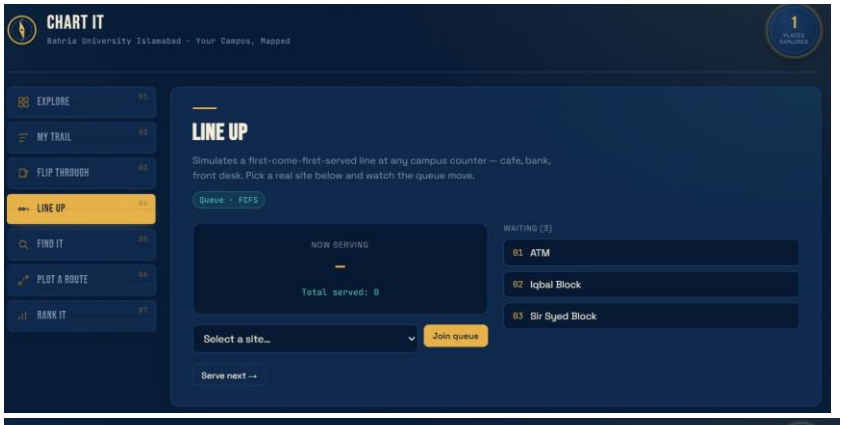


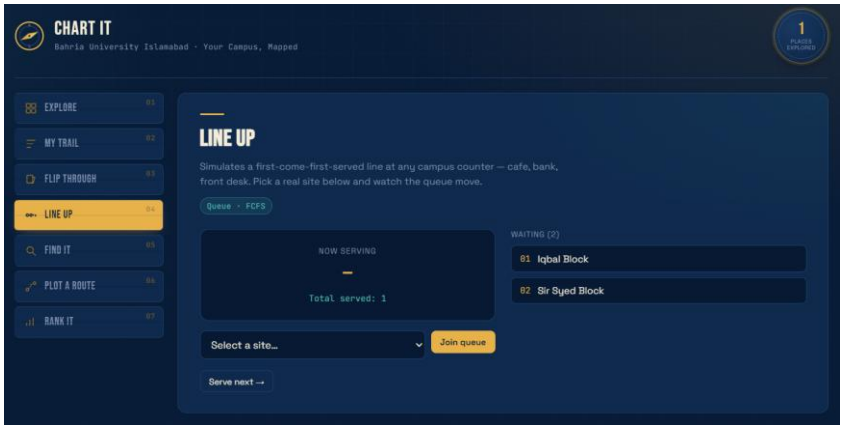
- **Visiting locations, viewing the log, and undo (Tasks 2 & 3)**



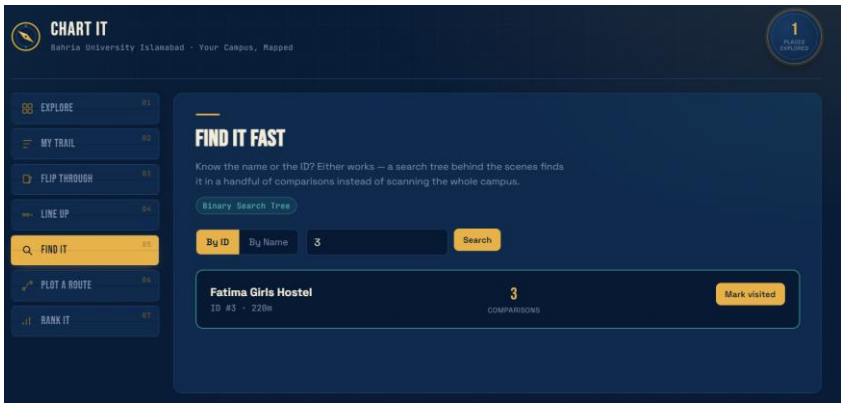


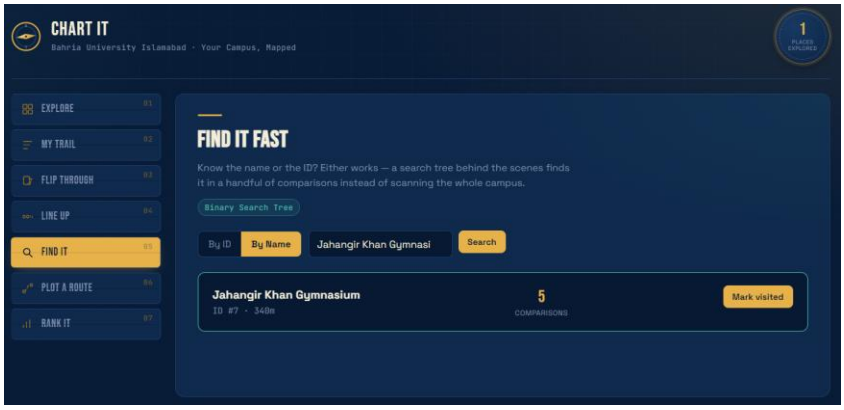
- **Busy resource queue simulation (Task 5)**



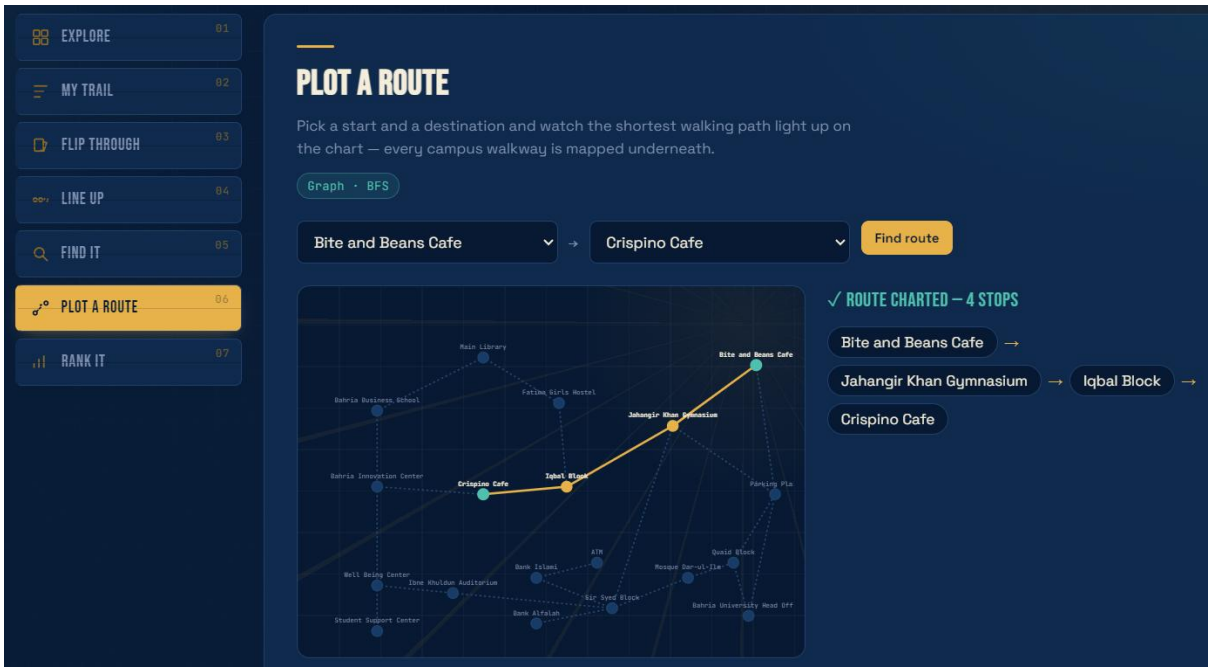


- **BST search by ID and by Name (Task 6)**

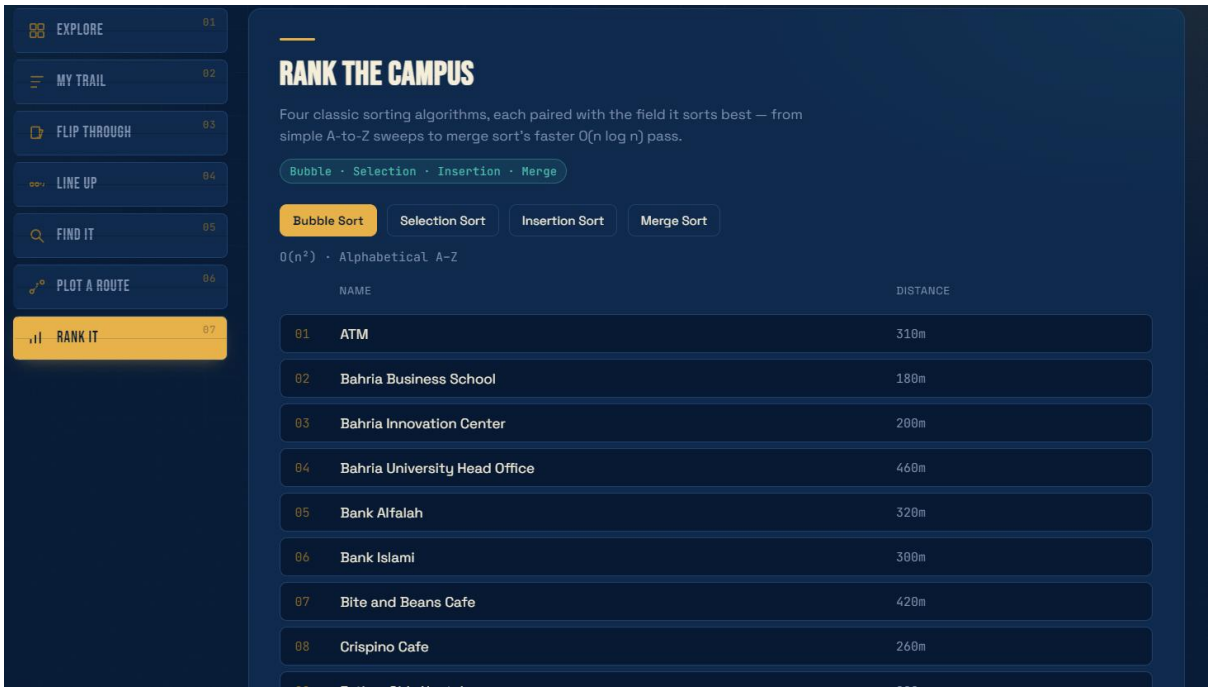




- **Reachability check with BFS path (Task 7)**



- **Sorting (Tasks 8 & 9)**



5. Conclusion

Campus Navigator demonstrates that a handful of classical data structures — arrays, stacks, queues, binary search trees, and graphs — are sufficient to build a genuinely useful campus tool. Each structure was chosen because it is the natural fit for its task: a stack for most-recent-first history, a queue for a real FCFS waiting line, a BST for fast lookup, and BFS over a graph for shortest-hop reachability. Building the dataset from the real BUIC campus guide map, rather than placeholder data, kept the project grounded in something concrete and verifiable rather than abstract. The result is a compact, dependency-free console application that is both a practical mini-project and a clear demonstration of core DSA concepts in action.